CafeOBJ

Especificação algébrica, reescrita de termos e demonstração por proof scores

Uma introdução prática para usar CafeOBJ como linguagem de especificação e verificação formal.

1. O que é CafeOBJ?

Uma linguagem de especificação algébrica e verificação formal

Ideia central

Descrever estruturas matemáticas por sorts, operadores e equações.

Usar a reescrita de termos como semântica operacional.

Executar especificações para reduzir termos a formas normais.

Para verificação

Uma propriedade pode ser demonstrada por reduções.

Provas mais complexas são organizadas como proof scores.

O ambiente também oferece suporte a indução, casos e busca de lemas.

2. Estrutura básica de uma especificação

```
mod! NAT {
   [ Nat ]
   op 0 : -> Nat {constr}
   op s : Nat -> Nat {constr}
   op _+_ : Nat Nat -> Nat
   vars N M : Nat
   eq 0 + N = N .
   eq s(N) + M = s(N + M) .
}
```

mod! → módulo estrito | [Nat] → sort | op → assinatura | eq → equação

3. Sintaxe: sorts, operadores e variáveis

Sorts

[Nat]

[NzNat < Nat]

Subsortes expressam uma relação de inclusão/tipagem.

Operadores

op 0 : -> Nat

op s : Nat -> Nat

op _+_ : Nat Nat -> Nat

Os underscores indicam as posições dos argumentos.

4. Equações e atributos de operadores

```
op _+_ : Nat Nat -> Nat {assoc comm}
op _*_ : Nat Nat -> Nat {assoc comm}
op e : -> G
op _*_ : G G -> G {assoc}
```

eq l = r . define uma equação/axioma que pode orientar a reescrita.

Atributos como assoc e comm informam propriedades estruturais do operador.

prec, l-assoc e r-assoc controlam a interpretação sintática.

5. De equações para regras de reescrita

A computação em CafeOBJ pode ser vista como redução de termos

```
eq 0 + N = N .
eq s(N) + M = s(N + M) .

s(s(0)) + s(0)
          ↓
s(s(0) + s(0))
          ↓
s(s(s(0) + 0))
          ↓
s(s(s(0)))
```

A redução procura subtermos que casam com o lado esquerdo de uma regra e os substitui pelo lado direito.

6. Redução de termos no interpretador

```
open NAT .
red s(s(0)) + s(0) .
close

-- resultado esperado:
-- s(s(s(0)))
```

open seleciona/abre o módulo para interação.

red reduz o termo usando as equações disponíveis.

Se a redução termina, o resultado é uma forma irreduzível (normal).

7. Igualdade equacional × igualdade por redução

= (teoria)

Aparece em equações: eq t = t' .

Expressa igualdade na teoria equacional especificada.

É a base das regras de reescrita executáveis.

== (avaliação)

Predicado de igualdade definido por avaliação.

Para termos ground, vale quando as reduções chegam a um termo comum.

É útil dentro de propriedades que serão reduzidas.

8. Demonstração: o que é um proof score?

Uma prova é construída combinando reduções e passos de raciocínio

```
open NAT .
  op n : -> Nat .
  op m : -> Nat .

  eq n = n .
  eq m = m .

  red n + m = n + m .
close
```

Um proof score é um roteiro executável de prova.

Constantes abertas representam elementos arbitrários.

Uma redução que termina em true descarrega o objetivo correspondente.

9. Exemplo clássico: comutatividade da soma

A ideia é provar ∀n,m. n + m = m + n

```
mod! NAT {
  [ Nat ]
  op 0 : -> Nat {constr}
  op s : Nat -> Nat {constr}
  op _+_ : Nat Nat -> Nat
  eq 0 + N = N .
  eq s(N) + M = s(N + M) .
}

-- Objetivo:
-- ∀n,m : Nat . n + m = m + n
```

A simples redução não basta para todos os pares de naturais.

É necessário introduzir um lema sobre m + 0 e, em seguida, usar indução.

10. Indução e estrutura de uma prova

Esquema matemático

Base: provar P(0).

Passo: assumir P(n) e provar P(s(n)).

Conclusão: ∀n. P(n).

No CafeOBJ

Definem-se constantes arbitrárias para representar os elementos.

Usam-se equações/lemmas para transformar o objetivo.

Comandos de prova podem organizar base, passo indutivo e subobjetivos.

11. Proof scores: um roteiro prático

```
open MODULO .
  -- declarar constantes arbitrárias
  -- introduzir hipóteses / lemas
  -- reduzir o objetivo
  red OBJETIVO .
close
```

1. Escolha o módulo adequado.
2. Transforme variáveis universais em constantes arbitrárias.
3. Adicione hipóteses como equações locais quando necessário.
4. Reduza o objetivo.
5. Se não fechar: faça casos, introduza lemas ou use indução.

12. Cuidados: quando uma redução realmente prova algo?

A especificação precisa representar corretamente a teoria pretendida.

As equações usadas na redução devem produzir uma computação adequada (por exemplo, evitar loops de reescrita).

Para um proof score complexo, o usuário é responsável por justificar a correção do roteiro de prova.

A redução de termos é uma ferramenta de raciocínio equacional — não apenas uma calculadora.

13. Além da reescrita equacional

Recursos importantes

Módulos protecting / extending / importing.

Equações condicionais: ceq.

Transições e reescrita de estados.

Hidden sorts e behavioral logic para objetos cujo comportamento define identidade.

Ferramentas de prova

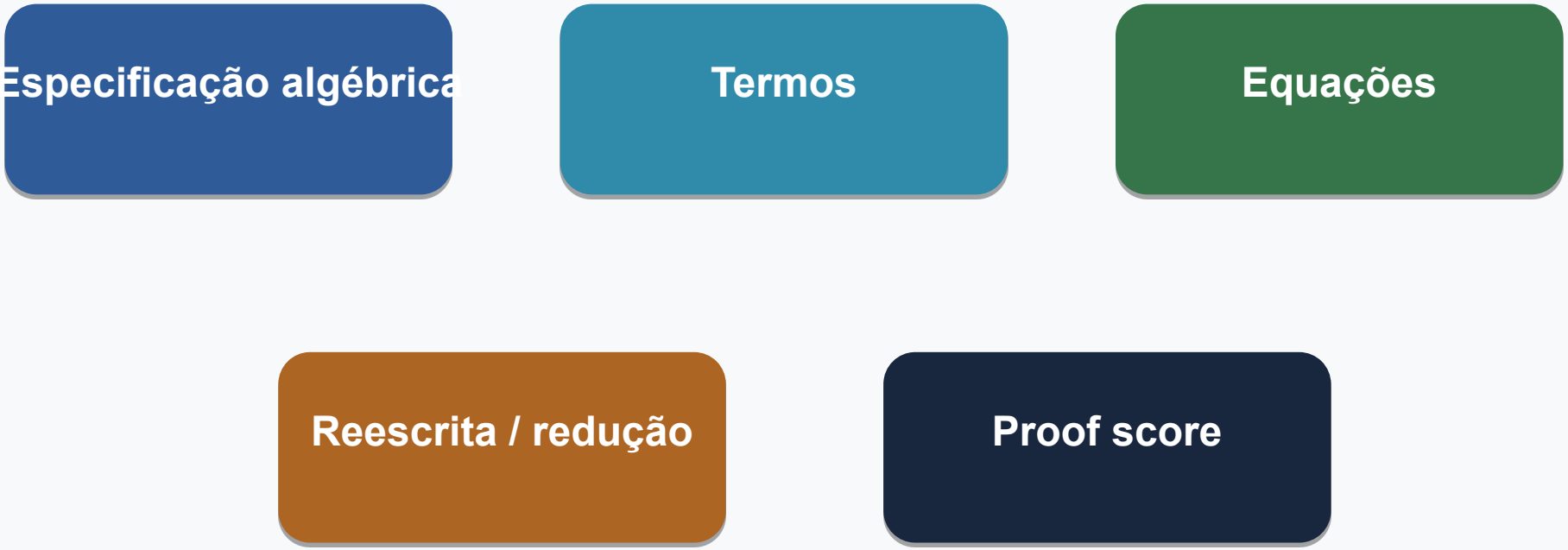
Proof scores.

Indução.

Case splitting.

CITP e comandos automáticos de apoio à demonstração.

14. Mapa conceitual



A especificação define a linguagem dos termos e suas equações; a reescrita executa a teoria; o proof score organiza reduções e raciocínios adicionais para demonstrar propriedades.

15. Referências e documentação

Sawada, T.; Futatsugi, K.; Preining, N. — CafeOBJ Reference Manual (v1.5.7).

CafeOBJ — Documentation, User Manual e tutoriais oficiais.

CafeOBJ User's Manual — capítulo sobre Term Rewriting Systems e avaliação/redução.

Introdução oficial — Proof scores.

```
cafeobj.org/documentation/
cafeobj.org/files/reference-manual.htm
l
cafeobj.org/files/manual.pdf
cafeobj.org/intro/en/proofscores.html
```